

ENCE 3210 – Microprocessors 1

Lab 4

February 27th, 2026

Dead Line: March 6th, 2026

Directions: Create a GitHub repository for this assignment and upload your answers there. Provide the GitHub link on the respective Canvas assignment.

Q1 – The code below was given to you by a firmware engineering company that wants you to improve it. The main goal is to simulate in software a 14-bit SAR and test the conversion for the following analog voltage levels: 0.42V, 0.83V, 1.65V, and 2.752V with a reference voltage of 3V.

```
1  #define bitsize 12
2
3  float Vref = 3.6;
4  float Vin = 3.3;
5  float thresh;
6  float quantized = 0;
7  int count;
8  int bitval;
9  int bits[bitsize];
10
11 void main (void){
12
13     Vref /= 2;
14     thresh = Vref;
15
16     for(count=0; count<bitsize; count++){
17         Vref /= 2;
18         if (Vin >= thresh){
19             bitval = 1;
20             thresh += Vref;}
21         else {
22             bitval = 0;
23             thresh -= Vref;}
24
25         bits[count] = bitval;
26         quantized += 2*Vref*bitval;}
27
28     while(1);
29 }
```

Q2 – You are responsible to design a solar charge controller code using your embedded platform with the specifications below:

- a) The system has a battery, a solar panel, a microcontroller, and a controllable switching element (such as a power multiplexer or analog switch IC).
- b) If the voltage output of the solar panel is greater than 2.5V, then the microcontroller power is supplied from the solar panel.
- c) If the voltage level of the solar panel is lower than 2.5V, then the battery will supply power to the microcontroller.
- d) If the voltage level of the battery drops below the voltage of the solar panel, then the switching element connects the battery to the solar panel.
- e) Provide a block diagram of this system.
- f) For software purposes, since we don't have access to a controllable switching element, use the OLED display to indicate how the power is routed in the system.

Q3 – The solar charge controller specifications in Q2 have a major issue, the system never has the chance to charge the battery fully.

- a) Improve the code from Q2 to allow charging the battery fully before switching the solar panels back to the microcontroller.
- [Note: if for some reason your solution in Q2 addresses this issue, disregard this question]

Q4 – Design a fan controller using your embedded platform with the specifications below:

- a) The system has the following components: an analog temperature sensor, a fan, and 2 user buttons.
- b) The temperature sensor will be monitored using the ADC module. You will measure 100 samples over a 5 second period and calculate the average temperature to find a reliable value.
- c) Then generate a PWM signal to change the speed of the fan according to the calculated temperature value. Set the PWM frequency as 250 Hz.
- d) Use button 1 to turn on or off the fan.
- e) Use button 2 to change the sensitivity of the fan speed with respect to temperature. Use three sensitivity levels such that the lower the sensitivity the lower the speed of the fan.
- f) Since the complexity of your code starts to increase, provide a code diagram of your solution.
- g) For software purposes, since we don't have access to a motor/fan, use an LED that is connected to a pin with PWM capabilities. The brightness of the LED will simulate the speed of the fan.